Advanced Machine Learning for Physics A.A. 2025/2026

Mapping physicochemical features onto 2D interface planes for protein–protein binding affinity prediction

Report di progetto – Documentazione tecnica della pipeline (Task 1–4)

Chiara Ritorto e Gianmarco Picchianti

20 ottobre 2026

Sommario

Questo report descrive il lavoro svolto nell'ambito del progetto proposto da E. Milanetti e G. Ruocco per il corso Advanced Machine Learning for Physics, finalizzato alla predizione dell'affinità di legame proteina–proteina a partire da mappe 2D di complementarità costruite tramite espansione in polinomi di Zernike, usate come input di una rete neurale convoluzionale (CNN). I risultati di questa pipeline sono confrontati con una seconda architettura, una CNN 3D che elabora direttamente le geometrie grezze delle interfacce senza passare dal descrittore di Zernike, a soddisfare il confronto con un modello di geometric deep learning richiesto dalla traccia.

Indice

1 Introduzione **2**

2 Dataset **2**

3 Task 1 – Architettura della pipeline **3**
3.1 Lettura del dataset . . . 3
3.2 Parsing delle strutture e download . . . 3
3.3 Identificazione dell'interfaccia . . . 4
3.4 Estrazione delle patch e generazione della superficie . . . 4
3.5 Orchestrazione e output . . . 4

4 Post-elaborazione: selezione dei dimeri e pulizia degli output **5**
4.1 Selezione dei dimeri (`scripts/task1/filter_dimers.py`) . . . 5
4.2 Pulizia delle cartelle (`scripts/task1/clean_task1.py`) . . . 5

5 Risultati dell'esecuzione completa **6**
5.1 Esempio: complesso 1KTZ . . . 6
5.2 Statistiche aggregate sui complessi dimerici . . . 7

6 Task 2 – Mappatura della complementarità tramite descrittori di Zernike **8**
6.1 Nota metodologica: espansione 2D . . . 9
6.2 Algoritmo implementato . . . 9
6.3 Esecuzione su HPC: perché il costo è stato elevato . . . 9

7 Task 3 – Costruzione dei piani di complementarità 2D **11**
7.1 Proiezione PCA sul piano medio di interfaccia . . . 11
7.2 Assegnazione delle proprietà biofisiche ai punti . . . 12
7.3 Filtro energetico di Lennard-Jones . . . 13
7.4 Discretizzazione su griglia 32×32 e canali . . . 14

8 Task 4 – Predizione dell'affinità tramite CNN **16**
8.1 Target . . . 16
8.2 Architettura `InterfaceCNN` . . . 16
8.3 Protocollo di addestramento . . . 16
8.4 Risultati . . . 17

9 Task 4b – Confronto con una baseline di geometric deep learning (CNN 3D) **17**
9.1 Voxelizzazione dell'interfaccia . . . 18
9.2 Architettura `Interface3DCNN` . . . 19
9.3 Protocollo di addestramento . . . 19
9.4 Risultati . . . 19

10 Discussione e limiti **20**
10.1 Bilancio rispetto all'obiettivo del progetto . . . 21

11 Sviluppi futuri **23**

1 Introduzione

L'obiettivo generale del progetto è costruire un framework fisicamente motivato per la predizione dell'affinità di legame (K_D o ΔG) tra coppie di proteine, rappresentando l'interfaccia di legame come una mappa 2D multi-canale (complementarità geometrica, potenziale elettrostatico, idrofobicità) da utilizzare come input per una rete neurale convoluzionale (CNN).

2 Dataset

Il dataset di partenza è l'*Affinity Benchmark v5.5*, fornito in formato Excel/CSV, contenente per ciascun complesso proteina–proteina:

- l'identificativo PDB della struttura cristallografica;
- le catene appartenenti al *ligand* e al *receptor*;
- il nome delle due proteine coinvolte;
- la costante di dissociazione K_D (in M);
- metadati sperimentali (metodo di risoluzione strutturale, risoluzione in Å, temperatura, riferimenti bibliografici/PubMed).

Il file utilizzato conta 207 complessi. A titolo di esempio, la prima riga del dataset descrive il complesso `1KTZ` (TGF-beta / TGF-beta receptor, catena ligando A, catena recettore B, $K_D = 2.9 \times 10^{-7}$ M, risolto per diffrazione a raggi X a 2.15 Å).

3 Task 1 – Architettura della pipeline

La Task 1, propedeutica a tutte le fasi successive, consiste nell'identificazione delle interfacce di legame a partire da un dataset di complessi proteina–proteina e nell'estrazione delle rispettive patch di superficie molecolare. Il codice, così come organizzato nel repository del progetto, è diviso in un package di moduli riutilizzabili (`src/`) e uno script a riga di comando che li orchestra (`scripts/task1/run_interface_extraction.py`):

- `src/dataset.py` – lettura del dataset e costruzione dei record dei complessi;
- `src/pdb.py` – parsing/scrittura di file PDB e download automatico da RCSB;
- `src/interface.py` – identificazione dei residui di interfaccia tramite distanze atomiche;
- `src/surface.py` – generazione della superficie molecolare "sampler" con densità elettronica approssimata;
- `src/chimerax.py` – generazione della superficie molecolare tramite ChimeraX (superficie escludente il solvente) e suo filtraggio sui residui di interfaccia;
- `src/pipeline.py` – orchestrazione dell'intero flusso per ciascun record del dataset;
- `scripts/task1/run_interface_extraction.py` – interfaccia a riga di comando per l'esecuzione della pipeline (importa i moduli da `src/`).

Il flusso complessivo, per ciascun complesso del dataset, è il seguente:

1. lettura del record (PDB ID, catene ligando/recettore, K_D) da `src/dataset.py`;
2. recupero della struttura 3D (file locale o download da RCSB) e parsing degli atomi;
3. separazione degli atomi in catene ligando e catene recettore;
4. identificazione dei residui di interfaccia tramite cutoff di distanza atomica;
5. estrazione degli atomi appartenenti ai residui di interfaccia (patch);
6. generazione della superficie molecolare e annotazione con densità elettronica approssimata;
7. scrittura degli output (strutture, residui di interfaccia, superfici, metadati) per il complesso.

3.1 Lettura del dataset

Il modulo `src/dataset.py` legge il file Excel del benchmark e costruisce, per ogni riga valida, un oggetto `ComplexRecord` contenente PDB ID, catene di ligando e recettore, K_D e metadati testuali.

3.2 Parsing delle strutture e download

Il modulo `src/pdb.py` implementa un parser leggero del formato PDB a colonne fisse, che estrae per ciascun atomo posizione, elemento chimico, catena, numero e nome di residuo, scartando le molecole d'acqua e gestendo gli *altloc*. Le strutture mancanti localmente vengono scaricate automaticamente da RCSB (`https://files.rcsb.org`). Sono inoltre presenti funzioni di selezione delle catene di interesse e di scrittura di sottoinsiemi di atomi in nuovi file PDB, utilizzate per isolare ligando, recettore e le rispettive patch di interfaccia.

3.3 Identificazione dell'interfaccia

L'interfaccia di legame viene identificata in `src/interface.py` calcolando, per ogni coppia di atomi pesanti (ligando, recettore), la distanza euclidea e trattenendo i contatti con distanza inferiore a un cutoff (di default **5.0 Å**). Il calcolo è vettorizzato tramite `numpy`, processando gli atomi del ligando a blocchi (*chunk*) per limitare l'uso di memoria sulle matrici di distanza. Per ciascuna coppia di residui in contatto vengono registrati la distanza minima e il numero di contatti atomici, mentre l'insieme complessivo dei residui di interfaccia (lato ligando e lato recettore) viene restituito come due insiemi separati.

3.4 Estrazione delle patch e generazione della superficie

Una volta identificati i residui di interfaccia, gli atomi corrispondenti vengono estratti (`atoms_in_residues`) e scritti in file PDB dedicati (catene complete e patch di interfaccia, separatamente per ligando e recettore).

La superficie molecolare può essere generata con due backend alternativi, selezionabili da riga di comando:

Sampler (`src/surface.py`) – per ogni atomo della patch viene generato un set deterministico di punti sulla sfera (distribuzione di Fibonacci, di default 64 campioni per atomo) a distanza raggio di van der Waals + raggio della sonda (probe radius, default 1.4 Å, valore tipico dell'acqua). I punti "sepolti" da altri atomi dello stesso partner vengono scartati tramite un test di occlusione basato sui raggi atomici. I punti rimanenti sono annotati con una densità elettronica approssimata, calcolata come somma di contributi gaussiani pesati per numero di elettroni degli atomi vicini entro un raggio di cutoff (default 6.0 Å), con decadimento controllato da un parametro α (default 1.8).

ChimeraX (`src/chimerax.py`) – la superficie viene generata invocando ChimeraX in modalità *headless* (`-nogui`) sul comando `surface`, che calcola una superficie escludente il solvente (*solvent-excluded surface*) sull'intera catena, esportata in formato OBJ. I vertici OBJ vengono poi associati al residuo più vicino e annotati con la stessa densità elettronica approssimata usata nel backend sampler. La strategia adottata di default (`full_chain_then_filter`) calcola la superficie sull'intera catena e la filtra successivamente sui soli residui di interfaccia, per evitare artefatti di bordo che si avrebbero calcolando la superficie direttamente sulla patch isolata; è comunque disponibile una modalità alternativa (`interface_patch_legacy` nel codice) che opera direttamente sulla patch.

Il backend di default nella pipeline è ChimeraX con filtraggio "full-chain-then-filter".

3.5 Orchestrazione e output

Il modulo `src/pipeline.py` coordina l'intero processo per ciascun record del dataset, gestendo anche il fallimento silenzioso di singoli complessi (ad es. struttura mancante o catene non trovate), che vengono registrati come errori senza interrompere l'elaborazione del batch. Per ogni complesso elaborato con successo viene creata una sottocartella `outputs/task1/{indice}_{PDB_ID}/` contenente:

- `ligand_chains.pdb`, `receptor_chains.pdb` – catene complete;
- `ligand_interface_patch.pdb`, `receptor_interface_patch.pdb` – patch di interfaccia;
- `ligand_interface_residues.csv`, `receptor_interface_residues.csv` – elenco dei residui di interfaccia (catena, numero, codice di inserzione, nome residuo);

- `interface_contacts.csv` – contatti residuo–residuo con distanza minima e numero di contatti atomici;
- `ligand_surface_patch.csv`, `receptor_surface_patch.csv` – superficie molecolare annotata (coordinate, normali, densità, residuo di appartenenza);
- `metadata.json` – parametri usati, conteggi di atomi/residui/punti di superficie ed esito dell'elaborazione.

Al termine dell'esecuzione viene inoltre prodotto un `manifest.csv` riassuntivo con lo stato (*ok*/*error*) di ciascun complesso processato. L'intera pipeline è esposta tramite CLI (`scripts/task1/run_interface_extraction.py`), con parametri configurabili per cutoff di interfaccia, backend di superficie, numero di campioni per atomo, raggio della sonda e parametri del modello di densità.

4 Post-elaborazione: selezione dei dimeri e pulizia degli output

Poiché il progetto si concentra su interfacce a due sole catene (un ligando e un recettore), l'insieme dei complessi processati dalla pipeline è stato in seguito ristretto ai soli casi **dimerici** tramite due script di post-elaborazione, eseguiti successivamente alla Task 1 vera e propria.

4.1 Selezione dei dimeri (`scripts/task1/filter_dimers.py`)

Lo script legge nuovamente il file Excel del benchmark e individua i complessi dimerici imponendo che sia la colonna `Ligand Chains` sia la colonna `Receptor Chains` contengano un singolo carattere (una sola catena per lato, senza elenchi separati da virgole). Per ciascun PDB ID identificato come dimero, la cartella corrispondente viene cercata in `outputs/all` (l'output completo della pipeline su tutto il dataset) e copiata in `outputs/task1`, che diventa così la directory di lavoro contenente esclusivamente i complessi dimerici.

Nota: 121 righe, 120 cartelle. Applicando il criterio di selezione al dataset si ottengono 121 righe che soddisfano la condizione di dimericità, ma solo 120 cartelle distinte in `outputs/task1`. La discrepanza è dovuta al complesso `1NVU`, che compare due volte nel benchmark come due varianti puntiformi dello stesso complesso strutturale (catena ligando `Q` in una riga, `R` nell'altra, entrambe accoppiate alla stessa catena recettore `S`, con costanti di dissociazione sperimentali diverse: $K_D = 3.6 \times 10^{-6}$ M e 1.9×10^{-6} M rispettivamente). Poiché la pipeline della Task 1 indicizza l'output per PDB ID e non per riga del dataset, le due esecuzioni scrivono nella stessa cartella `outputs/task1/.../1NVU`, e la seconda sovrascrive la prima: il numero di cartelle fisiche su disco (120) è quindi quello corretto da usare come riferimento per il conteggio dei complessi effettivamente disponibili per le fasi successive del progetto.

4.2 Pulizia delle cartelle (`scripts/task1/clean_task1.py`)

Una volta popolata `outputs/task1` con i soli dimeri, un secondo script rimuove da ciascuna sottocartella tutti i file non essenziali alle fasi successive del progetto, mantenendo esclusivamente:

- `ligand_surface_patch.csv`, `receptor_surface_patch.csv` – superfici di interfaccia annotate;
- `ligand_interface_residues.csv`, `receptor_interface_residues.csv` – elenco dei residui di interfaccia;
- `metadata.json` – parametri e conteggi dell'elaborazione.

File intermedi come le strutture PDB delle catene complete, le patch in formato PDB, i file OBJ di ChimeraX e i contatti residuo–residuo vengono eliminati, riducendo lo spazio occupato e mantenendo in `outputs/task1` solo gli artefatti necessari come input per la Task 2 (mappatura Zernike).

5 Risultati dell'esecuzione completa

Il dataset di partenza (207 complessi) è stato elaborato interamente dalla pipeline di Task 1, ma il progetto si concentra esclusivamente sui **complessi dimerici** (una sola catena per ligando e una per recettore), selezionati con lo script `scripts/task1/filter_dimers.py` descritto in Sezione 4: applicando questo criterio si ottengono **120 complessi dimerici**, tutti elaborati con successo dalla pipeline (nessun errore di download, parsing o calcolo della superficie). Le statistiche riportate di seguito si riferiscono esclusivamente a questo sottoinsieme.

5.1 Esempio: complesso 1KTZ

A titolo illustrativo, la Tabella 1 riporta i conteggi prodotti dalla pipeline per il complesso `1KTZ` (TGF-beta / TGF-beta receptor, catena ligando A, catena recettore B) – uno dei dimeri del sottoinsieme – con cutoff di interfaccia a 5.0 Å e backend ChimeraX (`full_chain_then_filter`).

Tabella 1: Conteggi della Task 1 per il complesso 1KTZ.

Grandezza	Valore
Atomi totali	1493
Atomi ligando (catena A)	653
Atomi recettore (catena B)	840
Residui di interfaccia – ligando	9
Residui di interfaccia – recettore	13
Contatti residuo–residuo	30
Atomi patch ligando	90
Atomi patch recettore	101
Punti superficie completa – ligando	84 247
Punti superficie completa – recettore	98 215
Punti superficie filtrata (interfaccia) – ligando	12 528
Punti superficie filtrata (interfaccia) – recettore	12 875

I residui di interfaccia identificati sul lato ligando includono, tra gli altri, ARG25, LYS31, TRP32 e HIS34; sul lato recettore, LEU27, PHE30, ASP32 e SER49, coerentemente con quanto atteso per l'interfaccia TGF-beta / recettore. Ogni riga del file `*_surface_patch.csv` rappresenta un punto della superficie molecolare, con coordinate cartesiane (`x,y,z`), normale (`nx,ny,nz`), densità elettronica approssimata e riferimento al residuo di appartenenza – formato pensato per essere direttamente compatibile con il successivo step di espansione in polinomi di Zernike (Task 2).

Il confronto tra i due pannelli (Fig. 1) è illustrativo delle differenze quantitative già riportate in Tabella 1 e nella Sezione 5. La patch di 1KTZ occupa un volume relativamente compatto (estensione massima di circa $28 \times 24 \times 19$ Å tra ligando e recettore) ed è composta da circa 12 500–12 900 punti di superficie per lato, coerentemente con un'interfaccia piccola (9 e 13 residui). La patch di 2OZA, al contrario, copre un'area sensibilmente più estesa e presenta ∼87 700 punti sul ligando e ∼86 400 sul recettore – circa sette volte più densa – riflettendo un'interfaccia con quasi otto volte più residui coinvolti (79 e 70) e una struttura complessiva di dimensioni maggiori (1493 atomi totali per 1KTZ contro 5440 per 2OZA). Questo conferma visivamente che il numero di

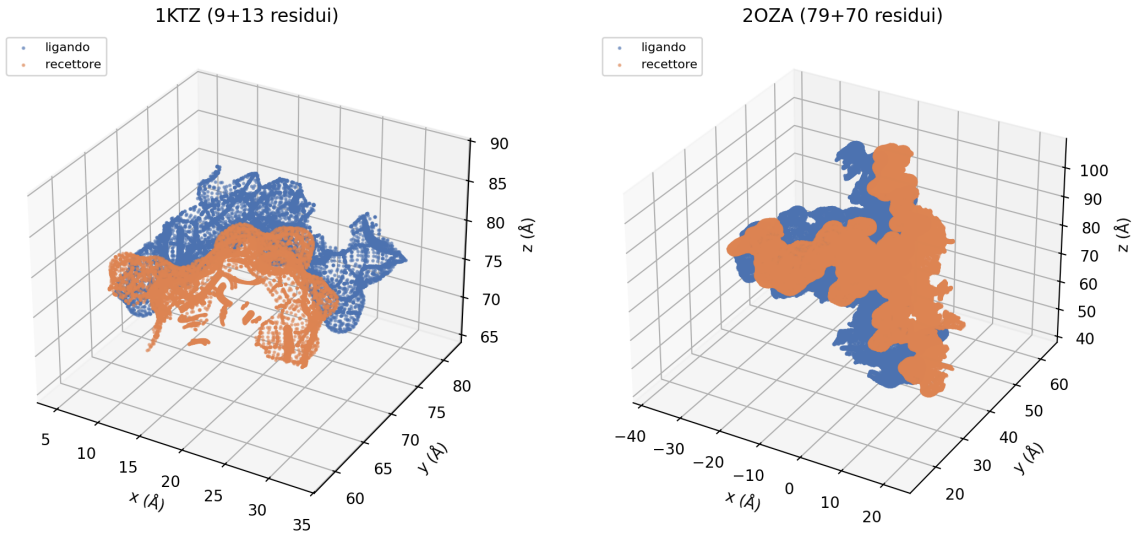


Figura 1: Visualizzazione 3D delle patch di superficie estratte per due complessi dimerici agli estremi opposti della distribuzione (assi in Å; in blu i punti della patch del ligando, in arancione quelli della patch del recettore). *Sinistra*: 1KTZ, l'interfaccia più piccola tra i dimeri selezionati (9 residui sul ligando, 13 sul recettore, 30 contatti residuo–residuo). *Destra*: 2OZA, l'interfaccia più estesa (79 residui sul ligando, 70 sul recettore, 209 contatti).

punti di superficie prodotti dalla pipeline scala con l'estensione reale dell'interfaccia biologica, e non è un artefatto dei parametri di campionamento (fissi per tutti i complessi).

5.2 Statistiche aggregate sui complessi dimerici

La Tabella 2 riassume la distribuzione delle grandezze principali calcolate dalla pipeline sui 120 complessi dimerici selezionati per il progetto. Le dimensioni dell'interfaccia (numero di residui e di contatti) e delle patch di superficie variano in modo considerevole anche all'interno di questo sottoinsieme, riflettendo l'eterogeneità dei sistemi dimerici presenti nel benchmark.

Tabella 2: Statistiche descrittive sui complessi dimerici selezionati per il progetto.

Grandezza	Media	Dev. std.	Minimo	Mediana	Massimo
Atomi totali	5268	3948	924	4370	24 125
Residui interfaccia – ligando	25.2	9.7	9	24	79
Residui interfaccia – recettore	22.7	9.7	10	21	70
Contatti residuo–residuo	62.7	24.0	27	61	209
Atomi patch – ligando	205.7	77.1	75	200	639
Atomi patch – recettore	185.1	79.9	69	169	579
Punti superficie (interfaccia) – ligando	32 894	19 572	10 754	26 551	109 741
Punti superficie (interfaccia) – recettore	27 298	14 236	12 323	23 162	99 169

Lettura dei risultati. Un'interfaccia tipica nel sottoinsieme coinvolge circa 20–25 residui per lato e ∼60 contatti residuo–residuo; la vicinanza tra media e mediana per tutte le grandezze indica distribuzioni ragionevolmente simmetriche. Il range è tuttavia ampio: il numero di residui di interfaccia varia da un minimo di 9 (ligando, complesso 1KTZ) fino a un massimo di 79 (ligando, complesso 2OZA), quasi un fattore 9 tra il caso più piccolo e il più grande del sottoinsieme. Il coefficiente di variazione (rapporto tra deviazione standard e media) è intorno al 35–40% per

la maggior parte delle grandezze, a indicare una dispersione elevata rispetto al valore medio: il "complesso dimerico medio" è quindi un'astrazione poco rappresentativa, e il dataset copre sistemi biologicamente eterogenei (presumibilmente dai piccoli complessi enzima–inibitore alle interfacce estese di tipo anticorpo–antigene, coerentemente con l'ambito applicativo descritto nella proposta di progetto).

Le grandezze scalano in modo internamente coerente: interfacce con più residui producono più atomi nella patch e, di conseguenza, più punti di superficie campionati, senza valori anomali che suggeriscano errori sistematici della pipeline (si veda anche il confronto diretto tra 1KTZ e 2OZA in Figura 1). Il rapporto tra punti di superficie e atomi della patch resta relativamente stabile tra i complessi, come atteso dato il numero fisso di campioni per atomo usato dal backend ChimeraX.

È inoltre utile notare che la dimensione complessiva del complesso (`all_atoms`) e la dimensione della sua interfaccia non sono strettamente correlate: 2OZA, pur avendo l'interfaccia più estesa del sottoinsieme, conta solo 5440 atomi totali, un valore vicino alla media, mentre il complesso più grande in assoluto (24 125 atomi) ha verosimilmente un'interfaccia proporzionalmente più contenuta rispetto alle sue dimensioni globali. Questo è rilevante per le fasi successive del progetto: la CNN della Task 4 riceve in input solo la patch locale di interfaccia (Task 3), non informazioni sulla proteina nel suo complesso, per cui la sua capacità predittiva dipende dalla geometria locale del sito di legame e non dalla dimensione della struttura ospite.

Infine, questa eterogeneità – combinata con la dimensione ridotta del campione (120 complessi) – è una delle cause plausibili della moderata capacità di generalizzazione osservata nei risultati della CNN (Sezione 8, $R \approx 0.42$): un dataset piccolo e disomogeneo nella forma delle interfacce rende più difficile per il modello apprendere pattern geometrici stabili e generalizzabili.

La Figura 2 mostra la distribuzione del numero di residui di interfaccia (per ligando e recettore) e del numero di contatti residuo–residuo sui complessi dimerici: la maggior parte delle interfacce coinvolge tra 15 e 35 residui per lato, con una coda di complessi ad interfaccia estesa (fino a ∼80 residui), coerente con la varietà di sistemi anticorpo–antigene ed enzima–inibitore presenti nel sottoinsieme.

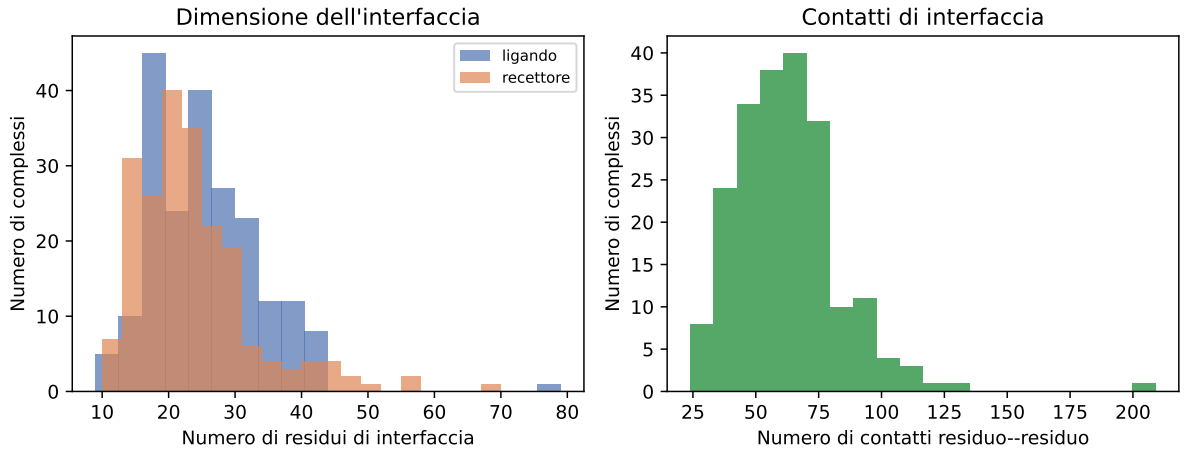


Figura 2: Distribuzione del numero di residui di interfaccia (sinistra) e del numero di contatti residuo–residuo (destra) sui complessi dimerici selezionati.

6 Task 2 – Mappatura della complementarità tramite descrittori di Zernike

Questa sezione è basata sulla lettura diretta dello script di produzione `run_task2_parallel.py`.

6.1 Nota metodologica: espansione 2D

La funzione utilizzata, `zepyros.get_zernike(coords, radius, center_index, order, verso)`, calcola il descrittore sulla patch di punti compresa entro un raggio fissato (`RADIUS = 6.0` Å) attorno a un singolo punto della superficie – non un vettore globale a dimensione fissa per l'intera proteina isolata. Il pacchetto `zepyros` (*ZErnike Polynomials analYsis of pROtein Shapes*) è stato sviluppato dallo stesso gruppo autore della proposta di progetto come motore di calcolo per il metodo descritto in Milanetti et al. [1] e nel webserver Zepyros [2], entrambi basati esplicitamente su un'espansione in **polinomi di Zernike 2D** di patch di superficie locali. La scelta implementativa di questo progetto è dunque pienamente coerente con quanto richiesto dalla traccia (Task 2: *"Expand patches in 2D Zernike polynomials"*), senza scostamenti metodologici da dichiarare.

6.2 Algoritmo implementato

Per ciascun complesso, a partire dalle patch di superficie prodotte dalla Task 1 (`ligand_surface_patch.csv`, `receptor_surface_patch.csv`):

1. i punti vengono sottocampionati con passo `STRIDE = 20` (indici `0, 20, 40, ...`);
2. per ciascun punto campionato del ligando si calcola il descrittore di Zernike locale (ordine `ORDER = 20`, raggio 6.0 Å), con orientazione `verso = +1`; per ciascun punto campionato del recettore, lo stesso con `verso = -1` (per mappare la corrispondenza concavità/convessità tra le due superfici);
3. si costruisce la matrice delle distanze euclidee *tra tutti* i descrittori del ligando e *tutti* i descrittori del recettore (`scipy.spatial.distance.cdist`) – si tratta quindi di un matching **globale nello spazio dei descrittori di forma**, non di un confronto ristretto alla coppia spazialmente più vicina;
4. per ogni punto del ligando si trattiene la distanza minima rispetto a tutti i punti del recettore (e viceversa per il recettore), definendo lo score complementarity_score $= -\min(\text{distanza})$: valori meno negativi indicano maggiore complementarità di forma;
5. i due versi (ligando→recettore e recettore→ligando) vengono uniti in un unico file, in modo che lo score complessivo non dipenda da quale dei due partner sia stato usato come riferimento.

L'output per complesso è `outputs/task2/{cartella}/zernike_complementarity_symmetric.csv`, con colonne `source` (ligand/receptor), `center_index`, coordinate `x,y,z` del centro, `zernike_distance` e `complementarity_score`. Nell'esempio del complesso 1KTZ, lo stride 20 applicato a 12 528 punti di superficie del ligando e 12 875 del recettore (Task 1) produce 627 centri campionati sul ligando e 644 sul recettore (1271 righe totali nel file di output), con punteggio di complementarità compreso nell'intervallo $[-3.73, -0.39]$.

Le Figure 3 e 4 riportano le stesse visualizzazioni generate nel notebook del progetto (Sezione 3, `PPB_Affinity.ipynb`). L'istogramma conferma che la distribuzione dello score non è patologica (nessun valore anomalo isolato, code contenute), mentre la visualizzazione spaziale mostra che i punti a complementarità più alta non sono distribuiti uniformemente sull'interfaccia ma si concentrano in specifiche regioni di contatto – coerentemente con l'aspettativa biologica che il legame sia guidato da un sottoinsieme di residui chiave piuttosto che dall'intera superficie di interfaccia in modo omogeneo.

6.3 Esecuzione su HPC: perché il costo è stato elevato

Il calcolo è stato eseguito su CINECA Leonardo con parallelizzazione a livello di complesso (`multiprocessing.Pool`, un processo per complesso), con il numero di processi sincronizzato al numero di core assegnati dallo scheduler (`SLURM_CPUS_PER_TASK`, di default 48 nello script).

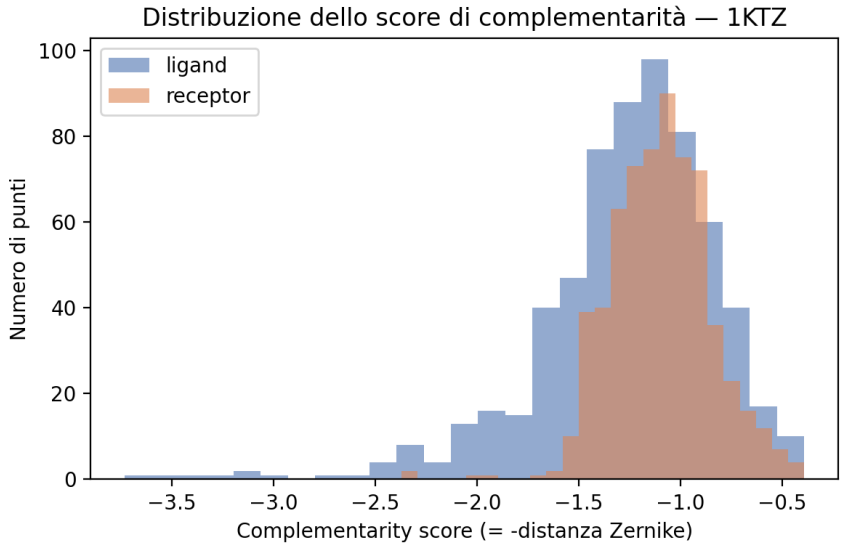


Figura 3: Distribuzione dello score di complementarità $-\min$(distanza Zernike) sui punti campionati del complesso 1KTZ, separatamente per ligando (627 punti) e recettore (644 punti). Le due distribuzioni sono centrate in modo simile (media -1.26 per il ligando, -1.10 per il recettore), con code verso valori più negativi (minore complementarità) leggermente più marcate sul lato ligando.

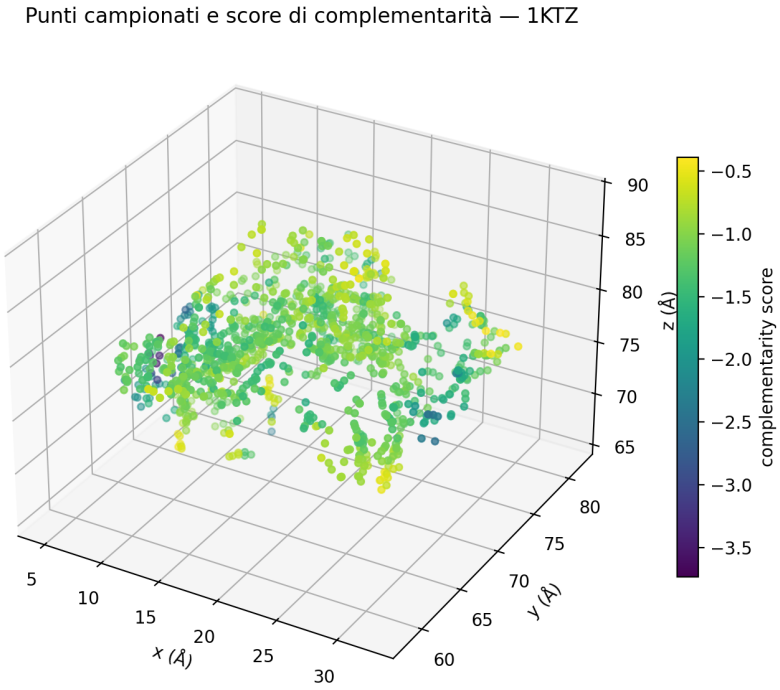


Figura 4: Posizione spaziale dei punti campionati per il complesso 1KTZ (ligando e recettore uniti), colorati in base al relativo score di complementarità (scala *viridis*: giallo = maggiore complementarità, viola = minore). La mappa evidenzia le zone dell'interfaccia in cui la forma locale delle due superfici combacia meglio secondo il criterio di Zernike, informazione che alimenta direttamente il canale di *shape complementarity* della Task 3.

Il consumo elevato è spiegabile con il concorso di due fattori, entrambi identificabili dalla struttura del codice:

1. **Costo computazionale intrinseco.** Per ciascun complesso, il matching di Zernike confronta *tutti* i descrittori campionati sul ligando con *tutti* quelli del recettore (`cdist`), un costo $O(n_{\text{lig}} \times n_{\text{rec}})$ nel numero di punti campionati. Poiché la dimensione delle interfacce nel dataset varia di quasi un fattore 9 (da 9 a 79 residui, si veda la Sezione 5), anche il numero di punti campionati – e quindi il numero di confronti da eseguire – varia in proporzione da un complesso all'altro: i dimeri con interfaccia più estesa richiedono un tempo di calcolo sensibilmente maggiore di quelli con interfaccia piccola come 1KTZ.

2. **Squilibrio di carico nella parallelizzazione.** La parallelizzazione è a grana grossa: un processo per complesso, senza suddivisione ulteriore del lavoro interno a un singolo complesso e senza bilanciamento dinamico del carico tra i processi. Questo significa che, se lo scheduler riserva un nodo con un numero fisso di core per l'intera durata del job, quel nodo resta occupato (e viene fatturato) fino a quando anche il processo più lento – tipicamente il complesso con l'interfaccia più estesa – non ha terminato, anche se la maggior parte degli altri processi ha già concluso da tempo.

A questi due fattori strutturali si è aggiunto un elemento pratico: un primo tentativo di esecuzione non ha completato il calcolo entro il limite di tempo richiesto alla coda Slurm ed è stato rilanciato con un tempo massimo maggiore. Lo script verifica, all'inizio dell'elaborazione di ciascun complesso, se il file di output esiste già (`if output_file.exists(): return`) e in tal caso lo salta: questo evita di ricalcolare i complessi già completati con successo nel tentativo precedente, ma non restituisce le ore già consumate dal tentativo interrotto, che restano comunque addebitate al budget del progetto. La combinazione di questi fattori – costo computazionale intrinsecamente variabile, squilibrio di carico tra processi paralleli di durata molto diversa, e un tentativo iniziale non andato a buon fine – rende plausibile un consumo complessivo dell'ordine delle migliaia di core-hours per processare l'intero insieme di 120 complessi dimerici.

Esito finale. Anche dopo l'ultimo rilancio, **2 complessi su 120 non hanno completato il calcolo** (`2PCC` e `1EWY`: le rispettive cartelle di output esistono ma sono vuote). Il numero di complessi effettivamente disponibili all'uscita della Task 2, e quindi utilizzato per tutte le fasi successive del progetto (Task 3 e Task 4), è pertanto **118**, non 120.

7 Task 3 – Costruzione dei piani di complementarità 2D

Questa sezione è basata sulla lettura diretta di quattro script: `run_task3_projection.py` (proiezione PCA), `run_task3_lennard_jones.py` (filtro energetico e correlazione), `generate_interface_images.py` (costruzione dei tensori 2D) e `plot_lennard_jones_correlation.py` (grafico comparativo).

Come chiarito in Sezione 6, dei 120 complessi dimerici selezionati nella Task 1 solo **118** hanno un output completo della Task 2 (mancano `2PCC` e `1EWY`, le cui cartelle sono rimaste vuote). Tutte le elaborazioni descritte in questa sezione sono quindi condotte su questo sottoinsieme di 118 complessi.

7.1 Proiezione PCA sul piano medio di interfaccia

Per ciascun complesso, lo script `run_task3_projection.py` carica la nuvola di punti 3D dell'interfaccia prodotta dalla Task 2 (ligando e recettore uniti in un'unica nuvola di punti) e vi applica un'analisi delle componenti principali a 3 componenti (`sklearn.decomposition.PCA`). I primi due assi principali (PC1, PC2) definiscono il **piano medio di best-fit** dell'interfaccia, su cui ciascun punto viene proiettato; il terzo asse (PC3) misura la deviazione del punto dal piano, cioè

la rugosità locale. Per ciascun complesso viene calcolata la percentuale di varianza spiegata da PC1+PC2, usata come indice di planarità dell'interfaccia. I risultati (coordinate proiettate per punto, e un riepilogo per complesso) sono salvati all'interno di `outputs/task3/` rispettivamente in `projected_points/*_projected.csv` e `summary_geometrico_pca_task3.csv`.

Risultato. Sui 118 complessi, la varianza spiegata dal piano medio è in media $86.3\% \pm 3.8\%$ (minimo 74.9%, massimo 93.3%; Figura 5). Le interfacce del dataset sono quindi effettivamente ben approssimate da un piano bidimensionale nella quasi totalità dei casi, un risultato empirico che supporta direttamente l'assunzione centrale del progetto (rappresentare l'interfaccia come piano 2D anziché come volume 3D).

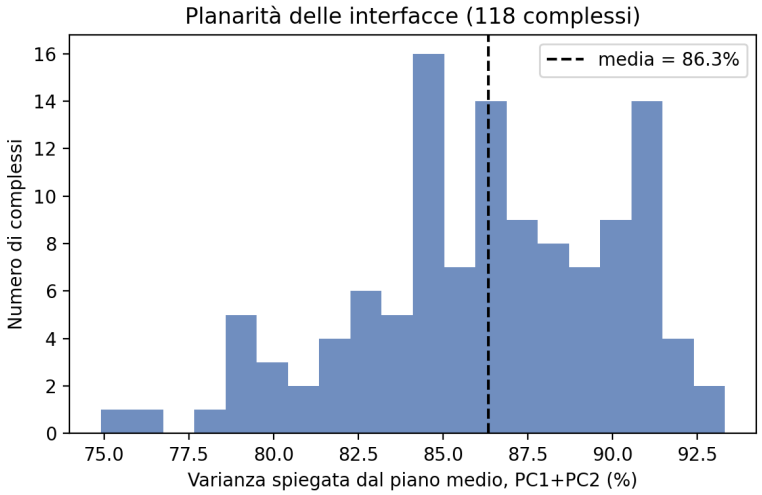


Figura 5: Distribuzione della percentuale di varianza spiegata dal piano medio di interfaccia (PC1+PC2) sui 118 complessi elaborati.

7.2 Assegnazione delle proprietà biofisiche ai punti

Per colorare la mappa con carica elettrostatica e idrofobicità, `generate_interface_images.py` deve associare a ciascun punto campionato un amminoacido. Va segnalato un limite metodologico non banale: il file di complementarità prodotto dalla Task 2 (`zernike_complementarity_symmetric.csv`, e la sua versione proiettata `*_projected.csv`) non conserva l'identità del residuo di origine di ciascun punto – contiene solo coordinate, indice del centro campionato e score di complementarità. Lo script assegna quindi gli amminoacidi **ciclando in ordine** sulla lista dei residui di interfaccia letta da `ligand_interface_residues.csv` / `receptor_interface_residues.csv` (un contatore separato per i punti di origine ligando e recettore, che scorre ciclicamente la lista dei residui, ripartendo dall'inizio ogni volta che la raggiunge in fondo). Si tratta quindi di un'approssimazione per interleaving, non di una vera mappatura spaziale punto–residuo: distribuisce comunque sull'interfaccia i valori chimici realmente presenti in quel complesso, ma senza garantire che un dato punto riceva il valore del residuo a cui è effettivamente più vicino nello spazio. È un'assegnazione approssimata ma non arbitraria, poiché usa comunque il pool corretto di amminoacidi del complesso in esame. Questo limite riguarda in particolare i Canali 2–3 definiti nella sezione successiva, che – a differenza del Canale 1 – non misurano una complementarità diretta tra ligando e recettore, ma la sola distribuzione (approssimata) delle proprietà chimiche di ciascun lato sulla mappa.

Le scale usate sono standard: carica elettrostatica formale netta a pH 7.4 (Arg/Lys $= +1$, Asp/Glu $= -1$, His $= +0.1$, tutti gli altri $= 0$) e idrofobicità di Kyte–Doolittle (da -4.5, Arg, a $+4.5$, Ile).

7.3 Filtro energetico di Lennard-Jones

Rispetto alla traccia, il progetto introduce un passaggio aggiuntivo di pesatura fisica dei punti di interfaccia (`run_task3_lennard_jones.py`). Per ciascun punto si calcola la distanza reale dall'atomo più vicino del *partner opposto*, usando le coordinate atomiche vere della patch di interfaccia (file `ligand_interface_patch.pdb` / `receptor_interface_patch.pdb`, letti con Biopython dalla cartella `all/` in cui la Task 1 li aveva originariamente scritti prima della pulizia di `clean_task1.py`), non le coordinate dei punti di superficie campionati. La logica è cross-interfaccia (due `KDTree` di `scipy` indipendenti: punti del ligando contro atomi del recettore e viceversa). A questa distanza si applica una funzione di peso ispirata al potenziale di Lennard-Jones ($\sigma = 3.5$ Å), nulla per $r < 2.0$ Å (regime repulsivo) e per $r > 6.0$ Å (interazione trascurabile), e compresa tra 0 e 1 nella buca attrattiva di van der Waals – la forma della funzione, verificata su un complesso di test, è mostrata in Figura 6 e corrisponde a quella attesa per un potenziale troncato.

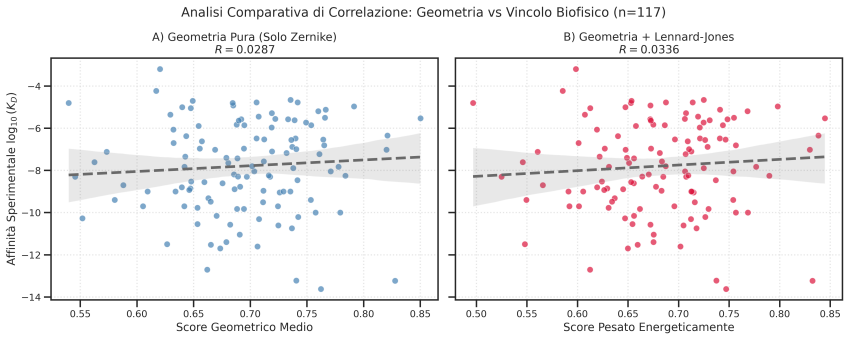


Figura 6: Correlazione tra lo score di complementarità geometrica (medio per complesso) e l'affinità sperimentale $\log_{10}(K_D)$, senza (pannello A) e con (pannello B) pesatura energetica di Lennard-Jones, sui 117 complessi disponibili dopo la correzione descritta di seguito.

Per ciascun complesso, lo score di complementarità *normalizzato* (`zernike_distance` riscalata in $[0, 1]$ **separatamente per ciascun complesso**, tramite Min-Max locale) viene mediato su tutti i punti campionati, sia in versione "pura" sia pesata dal peso di Lennard-Jones, e il risultato è correlato con $\log_{10}(K_D)$.

Correzione dati: duplicazione di `1NVU`. Il file di riepilogo prodotto dallo script (`pd.merge` tra gli score geometrici e il benchmark Excel, giunzione sul PDB ID) conteneva 118 righe per soli 117 PDB unici: `1NVU` compariva due volte, abbinato allo stesso identico score geometrico (calcolato una sola volta, sull'unica geometria disponibile su disco, verificata essere quella della catena ligando Q) ma a due valori di affinità sperimentale diversi – gli stessi che nel benchmark corrispondono alle catene Q e R (Sezione 8.4). È lo stesso problema di disallineamento geometria/target già identificato e corretto per la Task 4; qui è stato corretto rimuovendo la riga associata alla catena R, mantenendo quella coerente con la geometria realmente disponibile (catena Q). I risultati che seguono si riferiscono al dataset così corretto ($n = 117$).

Risultato (ricalcolato da `summary_lennard_jones_affinity.csv` dopo la correzione, $n = 117$):

	Pearson R	p-value	Spearman ρ	p-value
Geometria pura	0.029	0.759	0.081	0.383
Geometria + Lennard-Jones	0.034	0.719	0.088	0.346

Nessuna delle due correlazioni è statisticamente significativa ($p \gg 0.05$): a livello di media per complesso, né lo score geometrico puro né quello pesato energeticamente mostrano una relazione lineare o monotona rilevabile con l'affinità sperimentale su questo dataset. La correzione della

duplicazione di `1NVU` lascia il quadro sostanzialmente invariato rispetto alla stima preliminare (che includeva la riga duplicata, $R = 0.030$ e $R = 0.038$): con un segnale già così debole, un singolo campione ha un impatto trascurabile. Si tratta di un risultato negativo onesto, utile perché motiva concretamente il passaggio a un modello capace di sfruttare l'informazione spaziale completa dell'interfaccia (la CNN della Task 4) piuttosto che una sua media scalare, che a quanto pare non è sufficiente a catturare il segnale.

Nota metodologica. La normalizzazione Min-Max applicata *per singolo complesso* (non globalmente sul dataset) rende la media dello score non direttamente comparabile in scala assoluta tra complessi diversi. È plausibile che questo contribuisca all'assenza di segnale osservata qui.

7.4 Discretizzazione su griglia 32×32 e canali

Le coordinate proiettate (PC1, PC2) vengono interpolate su una griglia regolare 32×32 tramite `scipy.interpolate.griddata` (metodo `linear`, con fallback `nearest` per riempire eventuali celle non coperte dall'interpolazione lineare, ad esempio ai bordi della patch). Si generano tre canali:

Canale 1 – Shape complementarity: `complementarity_score` di Zernike (definito nella Task 2 come $-\min(\text{distanza})$, quindi ≤ 0): valori più vicini a 0 indicano *migliore* complementarità, valori più negativi indicano interfaccia geometricamente meno complementare in quel punto.

Canale 2 – Potenziale elettrostatico: carica formale netta del residuo assegnato (Sezione 7.2), proiettata sulla mappa – non una misura di complementarità chimica tra le due superfici. Valori teorici in $[-1, 1]$.

Canale 3 – Idrofobicità: valore Kyte–Doolittle del residuo assegnato, proiettato sulla mappa allo stesso modo del Canale 2. Valori teorici in $[-4.5, 4.5]$.

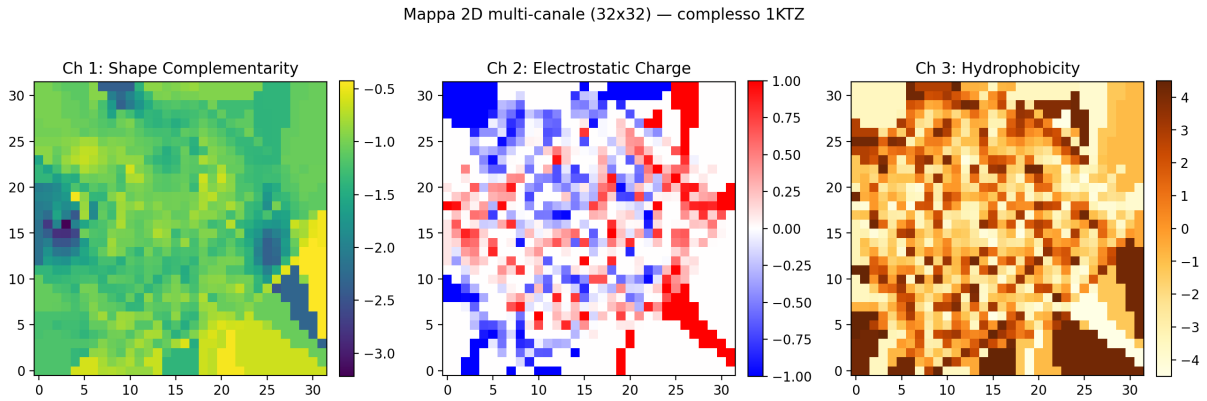


Figura 7: Mappa 2D multi-canale (32×32) generata per il complesso 1KTZ dopo la correzione del Canale 1: complementarità geometrica (sinistra, valori vicino a 0 = migliore complementarità), carica elettrostatica (centro), idrofobicità (destra).

A differenza di quanto talvolta assunto, questo script **non applica alcuna standardizzazione** ai valori: il tensore salvato contiene i valori fisici grezzi. Nell'esempio del complesso 1KTZ (Figura 7), il canale di shape complementarity varia tra -3.22 e -0.42 (media -1.16), il canale elettrostatico tocca l'intero range teorico $[-1, 1]$, e quello di idrofobicità l'intero range $[-4.5, 4.5]$ – prima di un'eventuale standardizzazione, canali con scale fisiche così diverse (uno centrato vicino a -1, uno vicino a 0, uno con deviazione standard ~ 2.8) entrerebbero nella CNN con pesi impliciti molto diversi, il che motiva la standardizzazione Z-score applicata più avanti nella pipeline di training (Sezione 8).

Per apprezzare la variabilità tra complessi diversi, la Figura 8 mostra le stesse tre mappe per altri tre complessi del dataset (`3SGB`, `1OC0`, `2SIC`), generate automaticamente ogni 20 complessi processati da `generate_interface_images.py`.

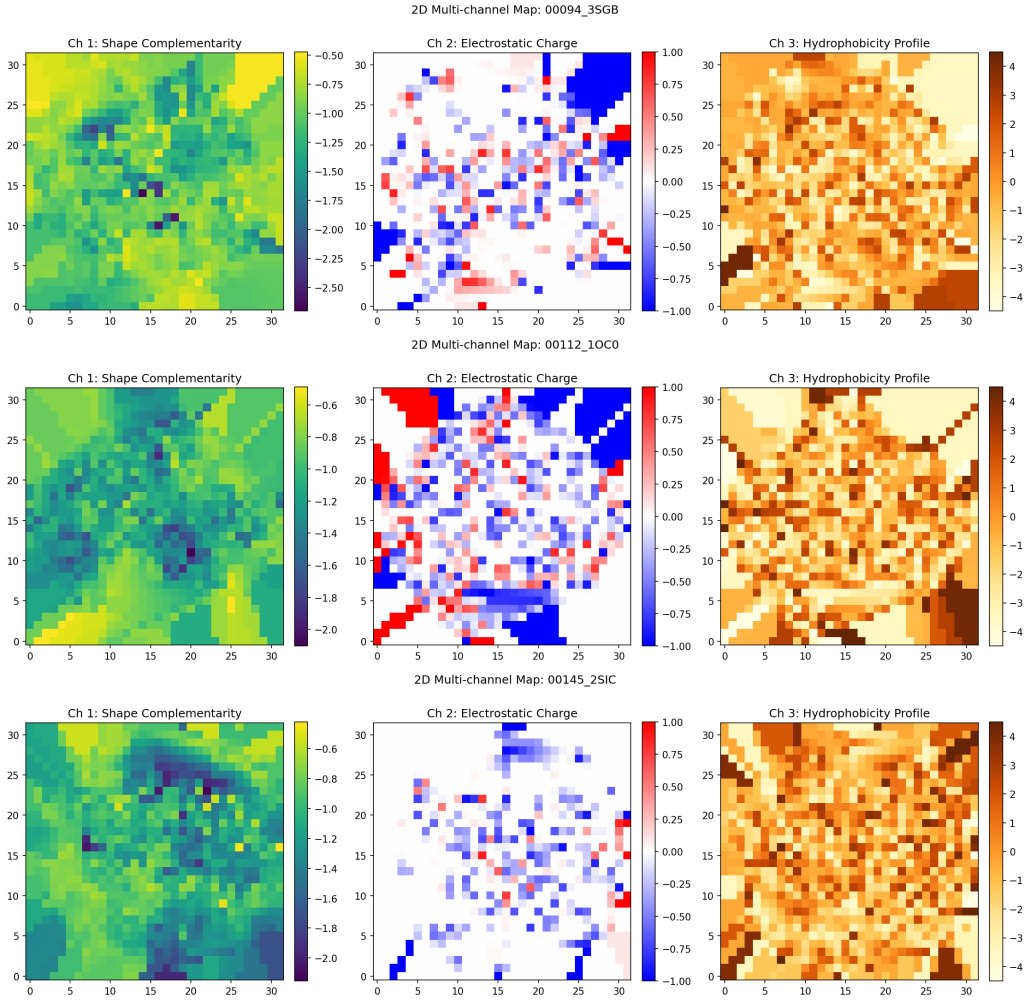


Figura 8: Mappe 2D multi-canale per tre ulteriori complessi del dataset: `3SGB` (sopra), `1OC0` (centro), `2SIC` (sotto).

Alcune osservazioni qualitative:

- **Canale 1 (shape complementarity):** in tutti e tre i casi la maggior parte della mappa è nei toni verde-giallo (valori tra -0.5 e -1.2 circa), con poche zone puntiformi molto più scure (fino a $-2/-2.5$) che indicano punti localizzati di complementarità particolarmente scarsa. Questo pattern – ampie zone di complementarità moderata con pochi "difetti" localizzati – è coerente con l'aspettativa biofisica che un'interfaccia di legame reale non sia uniformemente perfetta, ma presenti comunque regioni di incastro migliore o peggiore.

- **Canale 2 (carica elettrostatica):** appare visibilmente più *sparso* del canale 3 – gran parte della mappa è bianca (carica nulla), con macchie rosse/blu isolate. Questo riflette direttamente la scala di carica usata (Sezione 7.2): solo Arg, Lys, His, Asp e Glu hanno carica non nulla, mentre i restanti 15 amminoacidi standard contribuiscono zero. Si notano inoltre, in alcuni angoli delle mappe (es. 1OC0, in alto a sinistra e in alto a destra), blocchi compatti di colore uniforme: sono un artefatto dell'interpolazione `nearest` usata come fallback da `griddata` nelle regioni della griglia esterne all'inviluppo convesso dei punti campionati, dove il valore del punto valido più vicino viene esteso su un'area ampia.

- **Canale 3 (idrofobicità):** molto più "denso" e testurizzato del canale 2, perché quasi tutti gli amminoacidi hanno un valore di Kyte–Doolittle non nullo: la mappa presenta quindi una texture continua su tutta la superficie, senza le ampie zone bianche del canale elettrostatico.

L'output è un tensore `.npy` di forma $(3, 32, 32)$ per complesso, salvato in `outputs/task3/interface_maps/`, insieme a un'anteprima grafica a tre pannelli generata per un sottoinsieme dei complessi (uno ogni 20, per contenere il numero di immagini prodotte).

8 Task 4 – Predizione dell'affinità tramite CNN

Questa sezione è basata sulla lettura diretta di `models.py` e `train_final.py`.

8.1 Target

Il target di regressione è il logaritmo negativo della costante di dissociazione,

$$pK_D = -\log_{10} K_D,$$

scelta standard che linearizza un range di valori distribuito su diversi ordini di grandezza e lo rende proporzionale all'energia libera di legame.

8.2 Architettura `InterfaceCNN`

La rete (PyTorch, `models.py`) elabora tensori $(3, 32, 32)$ tramite tre blocchi convoluzionali a contrazione spaziale progressiva ed espansione dei canali latenti. Ogni blocco applica `Conv2d` (3×3, padding 1) $\rightarrow$ `BatchNorm2d` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2d`(2):

$$(3, 32, 32) \rightarrow (16, 16, 16) \rightarrow (32, 8, 8) \rightarrow (64, 4, 4)$$

Segue il blocco di regressione: `Flatten` $(1024) \rightarrow$ `Linear`$(1024,\ 128) \rightarrow$ `Dropout`$(p = 0.3) \rightarrow$ `Linear`$(128,\ 1)$, con uscita lineare non vincolata. La scelta di una rete custom, anziché di architetture pre-addestrate tipo ResNet o VGG, è motivata dal *mismatch* tra immagini naturali e griglie biofisiche 32×32 a canali non-RGB.

8.3 Protocollo di addestramento

Il protocollo usa una **procedura a due livelli** per ciascuno dei 5 fold esterni della cross-validation, pensata per evitare che qualunque decisione sul modello (statistiche di standardizzazione, scelta del checkpoint) sia influenzata dai dati usati per la valutazione finale di quel fold:

1. il training set esterno del fold viene ulteriormente suddiviso 85%/15% in un train-interno e un validation-interno (`sklearn.model_selection.train_test_split`, seed fisso);
2. la standardizzazione Z-score per canale viene calcolata **solo** sul train-interno, e applicata a tutti e tre i sottoinsiemi (train-interno, validation-interno, test esterno) del fold;
3. il modello viene addestrato sul train-interno; il checkpoint migliore viene scelto in base alla loss sul validation-interno (non sul test esterno);
4. il test esterno viene usato **esclusivamente** per generare le predizioni OOF finali, mai per alcuna decisione durante l'addestramento.

Il resto del protocollo:

- **Loss**: errore quadratico medio (MSE).
- **Ottimizzatore**: Adam, learning rate iniziale $\eta = 10^{-3}$, `weight_decay` $= 2 \times 10^{-3}$, con scheduler `ReduceLROnPlateau` (`factor` 0.5, `patience` 7 epoche, applicato sulla loss di validation-interna).
- **Data augmentation**: riflessioni orizzontali/verticali e rotazioni ortogonali discrete (90°, 180°, 270°) applicate ad ogni epoca, solo sul train-interno. La giustificazione è fisica: l'affinità è uno scalare termodinamico invariante per rotazione, mentre l'orientamento della griglia dipende da assi PCA arbitrari.
- **Hardware**: accelerazione su Apple Silicon tramite backend PyTorch MPS.
- 100 epoche, batch size 8, 5 fold esterni (`KFold`, seed 42).

8.4 Risultati

Dei 118 complessi con output completo della Task 3, altri due sono stati esclusi in questa fase dopo un controllo di integrità dei dati: `1NVU` (il complesso compare due volte nel benchmark originale con due catene ligando diverse, Q e R, entrambe accoppiate allo stesso recettore; il tensore su disco corrisponde solo a una delle due geometrie, verificata essere la catena Q, e per evitare un disallineamento tra input e target è stato escluso esplicitamente) e `1UUG` (la relativa costante di dissociazione nel benchmark è riportata come `<1E-13` M, un limite di rilevazione strumentale piuttosto che una misura puntuale, e non è quindi utilizzabile come target di regressione). Il dataset di training della Task 4 è pertanto composto da **116** complessi.

Predizioni out-of-fold aggregate sui 5 fold esterni:

	RMSE	Pearson R	p-value	Spearman ρ
CNN (`train_final.py`)	1.9913	0.4235	2.2×10^{-6}	0.4224

Il coefficiente di Pearson è altamente significativo ($p \ll 0.05$), con intervallo di confidenza al 95% (trasformazione di Fisher) pari a $[0.261, 0.562]$; il modello spiega circa il 17.9% della varianza ($R^2 = 0.179$). È un risultato modesto in valore assoluto ma **genuino**: nettamente superiore alla baseline geometrica pura della Task 3 ($R = 0.029$, non significativa, Sezione 7), a conferma che la mappa 2D completa contiene informazione predittiva che una sua media scalare non cattura. Rispetto al risultato preliminare ottenuto sui 118 complessi prima di questo controllo di integrità ($R = 0.388$), sia il coefficiente di correlazione sia la sua significatività migliorano leggermente: rimuovere due campioni con target inaffidabile o ambiguo (uno per disallineamento geometria/target, l'altro per censura sperimentale del dato) aiuta il modello più di quanto costi in termini di numerosità campionaria.

I pesi appresi da ciascuno dei 5 modelli (uno per fold, `cnn_final_fold_1.pth` … `cnn_final_fold_5.pth`) sono salvati come `state_dict` di PyTorch – 155 137 parametri totali per modello, distribuiti tra i tre blocchi convoluzionali, le statistiche di `BatchNorm` e i due layer densi finali. Insieme al file `oof_predictions.csv` (PDB ID, pK_D reale e predetto per ciascun complesso), permettono di rigenerare esattamente le predizioni OOF riportate sopra senza dover riaddestrare la rete, e costituiscono la base per eventuali analisi successive (ad es. mappe di salienza, Sezione 10).

9 Task 4b – Confronto con una baseline di geometric deep learning (CNN 3D)

La traccia richiede esplicitamente un confronto con un modello di geometric deep learning di riferimento (Sezione 10). Questa sezione descrive una seconda pipeline, indipendente dalla precedente, costruita per rispondere a questo requisito: una CNN 3D che elabora direttamente

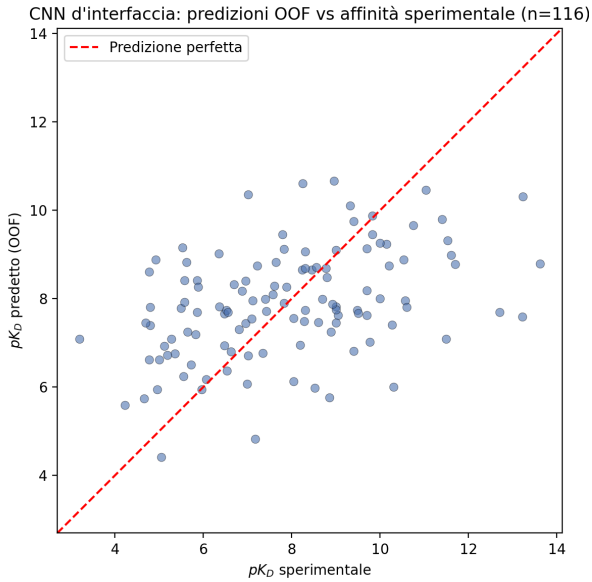


Figura 9: Predizioni out-of-fold della CNN (pK_D predetto) contro l'affinità sperimentale (pK_D reale) per i 116 complessi, con la linea di predizione perfetta $y = x$ come riferimento. La dispersione attorno alla diagonale è coerente con $R^2 \approx 0.18$: il modello cattura una tendenza reale ma con margini di errore ancora ampi caso per caso.

le patch di superficie della Task 1, **senza** passare né dal matching Zernike (Task 2) né dalla proiezione PCA sul piano medio di interfaccia (Task 3). È basata sulla lettura diretta di `scripts/task4b/voxelize_interface.py`, `models_3d.py` e `train_3d.py`.

9.1 Voxelizzazione dell'interfaccia

Per ciascun complesso, i file `ligand_surface_patch.csv` e `receptor_surface_patch.csv` prodotti dalla Task 1 vengono interpolati su una griglia regolare 16×16×16 (`scipy.interpolate.griddata`), analogamente alla discretizzazione 2D della Task 3 ma su un volume anziché su un piano proiettato. A differenza della Task 3, questi file conservano l'identità del residuo di origine di ciascun punto (colonna `residue_name`), il che permette qui una **vera mappatura spaziale punto–residuo** per i canali chimici, superando il limite di interleaving approssimato discusso in Sezione 7.2. Si generano 4 canali:

Canale 0 – Densità ligando: densità elettronica approssimata (Sezione 3) interpolata dai soli punti del ligando; posta a zero (non estesa per `nearest`) fuori dall'inviluppo convesso dei suoi punti, così da annullarsi fisicamente lontano dal ligando.

Canale 1 – Densità recettore: idem, dai soli punti del recettore.

Canale 2 – Carica elettrostatica: stessa scala della Task 3 (Sezione 7.2), calcolata sulla nuvola combinata ligando+recettore con fallback `nearest`.

Canale 3 – Idrofobicità: scala di Kyte–Doolittle, stesso trattamento del Canale 2.

A differenza del Canale 1 della Task 3 (uno score di complementarità già calcolato da Zernike), i Canali 0–1 qui sono le **due superfici tenute separate**: la rete riceve entrambe le geometrie nello stesso volume e deve apprendere autonomamente, tramite le convoluzioni 3D congiunte, dove le due forme si incastrano bene o male – non le riceve già confrontate a monte. È questo che rende la pipeline un confronto genuino con un approccio di geometric deep learning end-to-end, anziché una semplice riformulazione tridimensionale della Task 3.

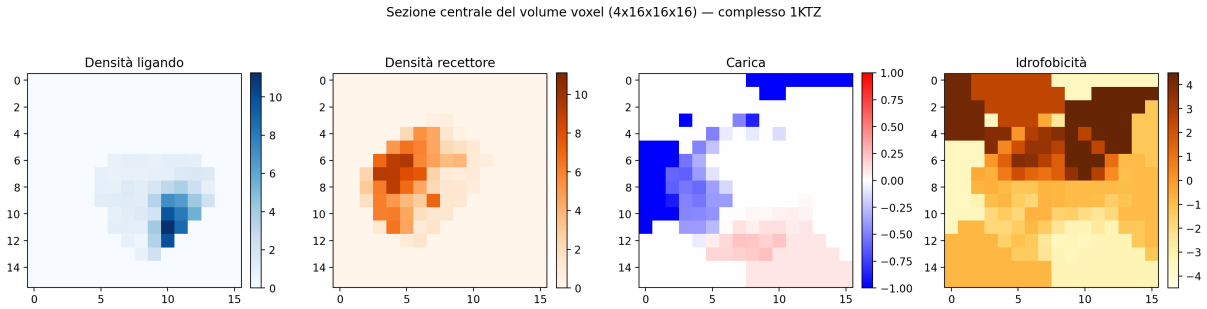


Figura 10: Sezione centrale del volume voxel $(4, 16, 16, 16)$ per il complesso 1KTZ. Le macchie di densità di ligando (blu) e recettore (arancione) sono geometrie distinte che si sovrappongono parzialmente nella zona di interfaccia.

Un vantaggio pratico di questa pipeline: non dipendendo dall'output della Task 2, recupera i 2 complessi (`2PCC`, `1EWY`) esclusi altrove per il collo di bottiglia computazionale del matching Zernike su Leonardo (Sezione 6.3). La voxelizzazione parte quindi da **120** complessi anziché 118.

9.2 Architettura `Interface3DCNN`

Analoga a `InterfaceCNN` (Sezione 8) ma con convoluzioni 3D: tre blocchi `Conv3d` ($3 \times 3 \times 3$, padding 1) $\rightarrow$ `BatchNorm3d` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool3d`(2):

$$(4, 16, 16, 16) \rightarrow (16, 8, 8, 8) \rightarrow (32, 4, 4, 4) \rightarrow (64, 2, 2, 2)$$

seguiti da `Flatten` (512) $\rightarrow$ `Linear`(512, 128) $\rightarrow$ `Dropout`($p = 0.3$) $\rightarrow$ `Linear`(128, 1) – 136 977 parametri totali, meno della metà dei 155 137 della CNN 2D nonostante il volume 3D, poiché il layer denso finale riceve una feature map spianata più piccola (512 contro 1024).

9.3 Protocollo di addestramento

Identico, riga per riga, al protocollo a due livelli della Task 4 (Sezione 8.3): stesso split interno 85/15 per la selezione del checkpoint, stessa standardizzazione Z-score calcolata solo sul train-interno di ciascun fold, stessa augmentation geometrica (qui estesa a riflessioni e rotazioni di 90° sui tre assi del volume), stesso ottimizzatore, stessi 5 fold esterni con seed 42. Un'unica differenza tecnica: le convoluzioni 3D non sono supportate dal backend MPS di PyTorch (confermato da un errore di runtime esplicito), per cui l'addestramento è stato eseguito su CPU (Google Colab) anziché su GPU Apple Silicon.

9.4 Risultati

Dei 120 complessi voxelizzati, valgono le stesse due esclusioni della Task 4 (`1NVU` per disallineamento geometria/target, `1UUG` per censura sperimentale del K_D, Sezione 8.4): il dataset di training è composto da **118** complessi, 2 in più della pipeline 2D grazie al recupero di `2PCC` e `1EWY`.

	n	RMSE	Pearson R	p-value	R^2
CNN 2D (`train_final.py`, Sez. 8.4)	116	1.9913	0.4235	2.2×10^{-6}	17.9%
CNN 3D (`train_3d.py`)	118	1.9790	0.4642	1.2×10^{-7}	21.5%

Il coefficiente di Pearson della CNN 3D ha intervallo di confidenza al 95% pari a $[0.309, 0.595]$. Numericamente, la CNN 3D è leggermente superiore su tutti i fronti – RMSE più basso, R e R^2 più alti, p-value più significativo – pur partendo da dati grezzi e senza alcun descrittore di forma

pre-calcolato a mano. Va però letto con la dovuta cautela statistica: gli intervalli di confidenza dei due R si sovrappongono ampiamente, per cui tale differenza non può essere considerata statisticamente significativa: i risultati indicano prestazioni comparabili tra le due architetture, non una netta superiorità della CNN 3D rispetto alla CNN 2D.

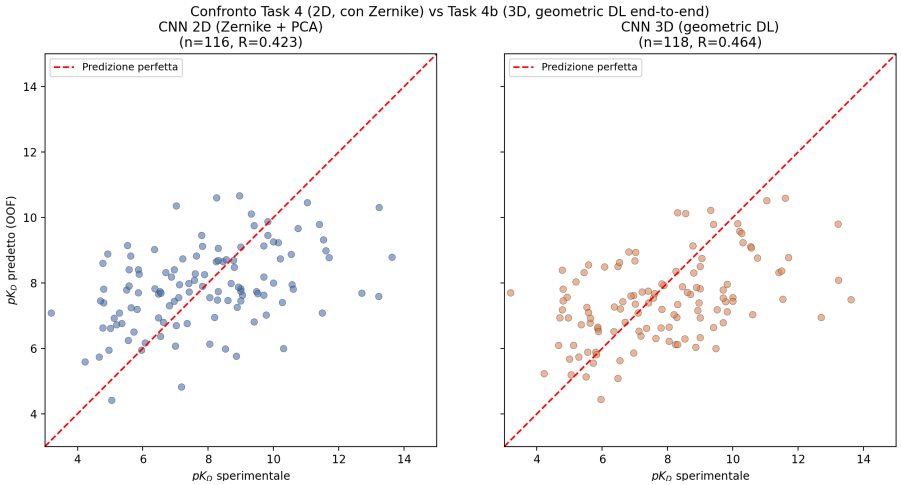


Figura 11: Predizioni out-of-fold della CNN 2D (sinistra, $n = 116$) e della CNN 3D (destra, $n = 118$) contro l'affinità sperimentale, sulla stessa scala di assi. Le due nuvole di punti mostrano una dispersione simile attorno alla diagonale di predizione perfetta.

Interpretazione. Il risultato ha valore scientifico indipendentemente da quale dei due modelli sia marginalmente migliore: dimostra che l'informazione geometrica rilevante per l'affinità può essere estratta da una rete 3D end-to-end direttamente dai dati grezzi della Task 1, senza il costoso passaggio intermedio di espansione in polinomi di Zernike (Task 2, eseguito su CINECA Leonardo con un consumo di migliaia di core-hours, Sezione 6.3). Questo soddisfa il confronto con un modello di geometric deep learning richiesto dalla traccia (Sezione 10).

10 Discussione e limiti

Dimensione del campione. Il limite dominante, comune a entrambi i modelli, è la numerosità. La CNN 2D lavora su 116 complessi (118 disponibili dalla Task 3, meno 2 esclusi per un controllo di integrità dei dati, Sezione 8.4): con $n = 116$, il coefficiente di Pearson $R = 0.4235$ ha un intervallo di confidenza al 95% pari a $[0.261, 0.562]$. La CNN 3D recupera 2 complessi in più ($n = 118$, Sezione 9.4), ma l'intervallo di confidenza del suo $R = 0.4642$ – $[0.309, 0.595]$ – resta di ampiezza confrontabile: il guadagno di due campioni non è sufficiente a stringere sensibilmente l'incertezza statistica. In entrambi i casi il segnale è reale e distinguibile da zero, ma un campione più numeroso resterebbe l'intervento più efficace per ridurre questa incertezza, indipendentemente dall'architettura scelta.

Confronti con baseline. La traccia richiede il confronto con modelli di geometric deep learning di riferimento e con descrittori Zernike puri. L'analisi di correlazione di Lennard-Jones (Sezione 7) costituisce di fatto la baseline "Zernike-only" richiesta dalla traccia: la media per complesso dello score geometrico (pesato o meno) non è significativamente correlata con l'affinità ($R \approx 0.03$–0.04, $p > 0.6$). Entrambe le CNN rappresentano un miglioramento sostanziale e statisticamente robusto rispetto a questa baseline – la 2D con $R = 0.4235$ ($p = 2.2 \times 10^{-6}$), la 3D con $R = 0.4642$ ($p = 1.2 \times 10^{-7}$) – a conferma che l'informazione spaziale completa dell'interfaccia contiene segnale non catturato da una sua semplice media scalare, indipendentemente da come

tale informazione viene rappresentata (piano 2D con descrittore Zernike pre-calcolato, o volume 3D grezzo). Il confronto con un modello di geometric deep learning di riferimento, richiesto dalla traccia, è coperto dalla CNN 3D della Task 4b (Sezione 9): le prestazioni risultano comparabili a quelle della pipeline 2D principale (la differenza tra i due R non è essa stessa statisticamente significativa, Sezione 9.4), pur partendo la CNN 3D da dati grezzi privi di qualunque descrittore di forma pre-calcolato a mano.

Cosa suggerisce il confronto 2D–3D sui descrittori di Zernike. Il fatto che la pipeline 2D (basata sui descrittori di Zernike pre-calcolati) e la CNN 3D end-to-end (senza alcun descrittore di forma pre-calcolato) raggiungano prestazioni statisticamente indistinguibili ($R = 0.4235$ vs. $R = 0.4642$, intervalli di confidenza ampiamente sovrapposti, Sezione 9.4) è un risultato che merita di essere letto anche nella direzione opposta a quella discussa sopra: una rappresentazione geometrica compatta come l'espansione in polinomi di Zernike 2D sembra conservare gran parte – se non la quasi totalità – dell'informazione geometrica rilevante per la predizione dell'affinità, senza che sia necessario elaborare direttamente la geometria voxelizzata completa in 3D. È un risultato potenzialmente interessante dal punto di vista del metodo: suggerisce che il descrittore di Zernike non comprima solo la rappresentazione dei dati, ma anche l'informazione realmente utile al compito. Va però letto con cautela: il confronto qui condotto è tra due *pipeline* complete (rappresentazione + CNN), non un confronto controllato che isoli il solo contributo del descrittore di Zernike a parità di tutto il resto – le due reti differiscono anche in architettura, canali di input e dimensione del dataset disponibile (116 vs. 118 complessi). Non si può quindi attribuire con certezza il risultato ai soli descrittori di Zernike; resta comunque un elemento promettente a favore della loro capacità di catturare l'informazione geometrica rilevante per il binding, che varrebbe la pena verificare più direttamente in un lavoro futuro (ad es. confrontando a parità di architettura un input Zernike-2D contro un input voxel-3D sullo stesso sottoinsieme di complessi).

Superficie molecolare: limite noto. Il comando ChimeraX usato in `chimerax.py` (Sezione 3) esegue `open`, `surface`, `save ... format obj` senza nascondere la rappresentazione atomica (sfere e bastoncini) prima dell'esportazione: una verifica a posteriori ha misurato che circa il 25% dei punti (mediana) risulta geometricamente interno a un atomo, quindi non appartiene alla superficie molecolare vera e propria. Questo intacca sia i conteggi di punti riportati in Tabella 1 e Tabella 2, sia l'affermazione in Sezione 5 secondo cui il numero di punti scala con l'estensione reale dell'interfaccia e non con i parametri di campionamento – affermazioni che restano quindi da intendersi come non verificate rispetto a questo effetto. La correzione è nota e localizzata (aggiungere `-cmd "hide atoms"` e `-cmd "hide cartoons"` prima di `surface`), ma richiede di rilanciare sia la Task 1 sia la Task 2 per rigenerare superfici, tensori e risultati a valle.

Interpretabilità. La traccia richiede una mappatura interpretabile tra feature di interfaccia e affinità. Un'estrazione di mappe di salienza tramite gradienti (calcolo del gradiente dell'output rispetto a ciascun pixel/voxel di input, per identificare le regioni della mappa che maggiormente influenzano la predizione) rappresenterebbe la via naturale per verificare se le zone più rilevanti per la CNN coincidono con residui biologicamente noti come hotspot di legame; non è stata eseguita in questa fase del progetto e resta elencata come sviluppo prioritario in Sezione 10.1.

10.1 Bilancio rispetto all'obiettivo del progetto

La proposta di progetto chiedeva di **predire quantitativamente quanto due proteine siano compatibili al legame** – cioè la loro affinità sperimentale K_D – a partire da una rappresentazione fisicamente interpretabile della morfologia della loro interfaccia (complementarità geometrica, elettrostatica, idrofobicità).

Cosa è stato effettivamente costruito. La pipeline realizza per intero la catena richiesta dalla traccia: identificazione dell'interfaccia e delle sue patch di superficie (Task 1), quantificazione della complementarità di forma tramite espansione in polinomi di Zernike 2D (Task 2), costruzione di piani di complementarità 2D multi-canale fisicamente interpretabili (Task 3), e un modello supervisionato (CNN) che apprende a predire pK_D da queste mappe (Task 4). A questa pipeline principale si affianca una seconda architettura indipendente (Task 4b), una CNN 3D end-to-end che soddisfa esplicitamente il confronto con un modello di geometric deep learning richiesto dalla traccia. Su questo piano puramente realizzativo, l'obiettivo della traccia è soddisfatto in tutte le sue componenti: esiste una pipeline riproducibile end-to-end, dal dato grezzo (struttura PDB) alla predizione numerica di affinità, corredata di una baseline di confronto genuina.

Quanto siamo vicini all'obiettivo predittivo. I due modelli spiegano rispettivamente il 17.9% (CNN 2D, $R^2 = 0.179$) e il 21.5% (CNN 3D, $R^2 = 0.215$) della varianza dell'affinità sperimentale sul dataset disponibile. Sono risultati **reali e non casuali** – la significatività statistica è netta in entrambi i casi ($p = 2.2 \times 10^{-6}$ e $p = 1.2 \times 10^{-7}$ rispettivamente) e il miglioramento rispetto alla baseline puramente geometrica (che non ha alcun potere predittivo rilevabile, $R \approx 0.03$) dimostra che l'informazione geometrica dell'interfaccia – rappresentata sia come mappa 2D con descrittore Zernike sia come volume 3D grezzo – contiene segnale informativo genuino sull'affinità. Ma è altrettanto vero che, in entrambi i casi, tra il 78% e l'82% della variabilità dell'affinità resta **non spiegato** dal modello: allo stato attuale nessuna delle due pipeline è in grado di predire con precisione pratica il valore di K_D di un nuovo complesso, né di discriminare in modo affidabile tra affinità simili. Per l'applicazione più ambiziosa menzionata nella proposta stessa – la Task 5 opzionale di affinity maturation in-silico, che richiederebbe di classificare correttamente il segno e l'entità di piccole variazioni di affinità dovute a singole mutazioni – un R^2 inferiore al 25% è insufficiente per entrambi i modelli: un simile utilizzo pratico richiederebbe un potere predittivo sostanzialmente maggiore, indipendentemente dall'architettura scelta.

Il valore dei risultati. Come dimostrazione che l'informazione geometrica dell'interfaccia contiene segnale predittivo reale, distinguibile dal rumore statistico e superiore a una baseline puramente geometrica – e che questo segnale è recuperabile sia da un framework fisicamente motivato (Zernike 2D + canali chimici + CNN) sia da un approccio di geometric deep learning end-to-end su dati grezzi – il risultato è positivo e onestamente riportabile, ed è rafforzato proprio dalla convergenza tra le due pipeline indipendenti. Come strumento pronto per un uso predittivo applicato, il risultato resta preliminare per entrambe.

Cosa servirebbe per migliorare, in ordine di impatto atteso.

1. **Più campioni.** È il fattore singolo più limitante, per entrambe le architetture. Con $n = 116$–118 e reti convoluzionali di $\sim$137 000–155 000 parametri, il rapporto tra numerosità del campione e capacità del modello è sfavorevole; gli intervalli di confidenza ampi su R (discussi sopra) sono il sintomo diretto di questo limite. L'intero benchmark PPB-Affinity originale contiene molte più strutture dei dimeri qui usati (il sottoinsieme è stato ristretto ai soli complessi a due catene per semplicità geometrica, e ulteriormente ridotto per controlli di integrità dei dati): estendere la pipeline ai complessi multi-catena sarebbe la strada più diretta per aumentare n senza cercare un dataset esterno, e beneficerebbe entrambi i modelli.

2. **Normalizzazione globale, non per-complesso, nel punteggio Zernike grezzo.** Come discusso in Sezione 7, la baseline geometrica usa una normalizzazione Min-Max calcolata separatamente per ciascun complesso, che verosimilmente cancella proprio l'informazione

di intensità assoluta più rilevante per l'affinità – lo stesso problema già risolto per l'input di entrambe le CNN passando a una standardizzazione Z-score globale.

3. **Assegnazione punto–residuo più accurata nella pipeline 2D (Sezione 7.2).** L'attuale schema di interleaving non conserva la posizione spaziale: una verifica sul complesso 1KTZ mostra che solo il 17% dei punti riceve il residuo effettivamente più vicino, per cui i Canali 2 e 3 riportano la composizione chimica dell'interfaccia distribuita sulla mappa, non la sua vera disposizione spaziale. L'informazione per una mappatura esatta è già disponibile: `center_index` (Task 2) indica la riga corrispondente nel file `*_surface_patch.csv` della Task 1, che contiene la colonna `residue_name` – la correzione richiede quindi solo di sostituire il contatore ciclico con una lettura diretta (`residue_name` alla riga `center_index` della patch del lato corrispondente) e rigenerare i tensori, un'operazione dell'ordine dei minuti, non delle ore. La Task 4b supera già questo limite per la propria rappresentazione voxel (Sezione 9), grazie ai dati della Task 1.

4. **Confronto con ulteriori baseline di geometric deep learning.** La CNN 3D della Task 4b (Sezione 9) soddisfa il requisito della traccia con un primo confronto genuino; resterebbe utile estenderlo a un secondo confronto con un'architettura pubblicata (es. una Graph Neural Network sui residui di interfaccia), per una contestualizzazione più ampia rispetto allo stato dell'arte.

5. **Verifica di robustezza: split per omologia di sequenza.** Una verifica condotta a valle del protocollo standard mostra che il 64% dei complessi nel test set di ciascun fold ha un omologo di sequenza nel training set (convenzione union) – una possibile fonte di ottimismo nella stima delle prestazioni se la CNN sfruttasse similarità di sequenza piuttosto che pattern geometrici generalizzabili. Un rilancio con fold costruiti per gruppi di omologia (nessuna proteina omologa condivisa tra training e test) produce però un risultato sostanzialmente invariato ($R = 0.37 \pm 0.03$): un'indicazione incoraggiante che il segnale appreso dal modello non dipenda in modo rilevante dalla presenza di omologhi tra training e test. Vale la pena riportare questa verifica in modo più prominente nei risultati (Sezione 8.4) in una revisione successiva.

6. **Interpretabilità tramite mappe di salienza**, per verificare se le regioni della mappa (2D o voxel 3D) su cui la CNN si concentra corrispondono a residui biologicamente plausibili (hotspot di legame noti in letteratura) – un controllo di sanità che, se superato su entrambe le architetture, rafforzerebbe sostanzialmente la credibilità di entrambi i modelli anche a fronte di un R^2 non elevato.

7. **Potenziale elettrostatico più realistico.** Il Canale 2 usa attualmente una scala di carica formale netta semplificata (Sezione 7.2); sostituirla con un calcolo del potenziale elettrostatico tramite APBS (*Adaptive Poisson-Boltzmann Solver*) – che tiene conto della distribuzione di carica reale, della costante dielettrica del solvente e degli effetti di schermatura ionica – produrrebbe una rappresentazione elettrostatica più realistica e potenzialmente più informativa per il modello.

11 Sviluppi futuri

La direzione naturale di estensione è il passaggio a una predizione *blind*, in cui il modello riceva le superfici complete delle due proteine isolate senza conoscere a priori la regione di legame. Questo trasformerebbe il lavoro da esercizio di regressione supervisionata su interfacce note a strumento di screening virtuale applicabile a coppie di proteine mai co-cristallizzate, ad esempio a partire da strutture predette.

Riferimenti bibliografici

[1] E. Milanetti, M. Miotto, L. Di Rienzo, M. Monti, G. Gosti, G. Ruocco. 2D Zernike polynomial expansion: Finding the protein-protein binding regions. *Computational and Structural Biotechnology Journal*, 19:29–36, 2021. https://doi.org/10.1016/j.csbj.2020.11.051

[2] M. Miotto, L. Di Rienzo, L. Bo', G. Ruocco, E. Milanetti. Zepyros: a webserver to evaluate the shape complementarity of protein–protein interfaces. *Bioinformatics Advances*, 5(1):vbaf051, 2025. https://doi.org/10.1093/bioadv/vbaf051